Reinforcement Learning from Human Feedback (RLHF)

Tirtharaj Dash

Department of CS & IS
BITS Pilani, Goa Campus, India
Homepage

April 2026

So far:

We have discussed two ways to make a model (LLM) behave well:

Supervised fine-tuning (SFT):

- give the model example input–output pairs
- train it to imitate those outputs
- works well when we have good demonstrations

Reinforcement learning (RL):

- agent gets a reward signal from the environment
- learns to take actions that maximise reward
- works well when reward is easy to define (e.g., game score)

What if neither approach alone is enough?

The problem with LLMs

Suppose we fine-tune a language model to answer user questions.

Problem 1: demonstrations are expensive.
Writing high-quality responses for every type of query is hard.

Problem 2: there is no obvious reward.
What is the "score" for a good answer?
Helpfulness? Safety? Honesty?
These are human judgements, not numbers.

Problem 3: the model can be unhelpful or even harmful.
A model trained only to predict text can produce confident nonsense, biased outputs, or unsafe content.

The problem with LLMs

Suppose we fine-tune a language model to answer user questions.

Problem 1: demonstrations are expensive.
Writing high-quality responses for every type of query is hard.

Problem 2: there is no obvious reward.
What is the "score" for a good answer?
Helpfulness? Safety? Honesty?
These are human judgements, not numbers.

Problem 3: the model can be unhelpful or even harmful.
A model trained only to predict text can produce confident nonsense, biased outputs, or unsafe content.

We need a way to incorporate human judgement into training.

Learning from human preferences

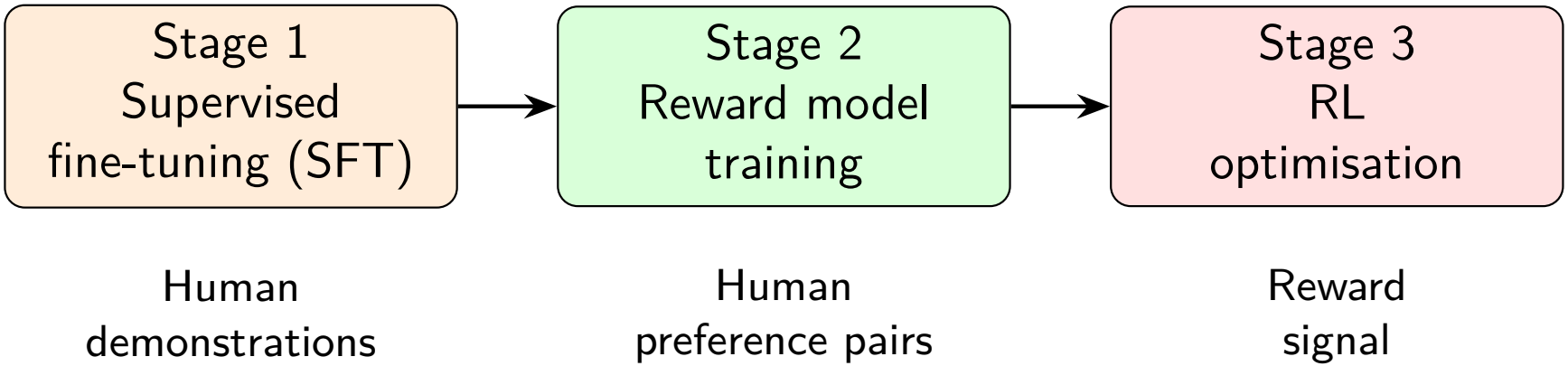
Instead of defining a reward function by hand:

1. Ask humans: *which of these two responses is better?*
2. Learn a reward model from those preferences
3. Use RL to make the LLM produce responses the reward model scores highly

This is Reinforcement Learning from Human Feedback (RLHF).

Christiano et al. (2017). Deep RL from Human Preferences. *NeurIPS*.

RLHF: three-stage pipeline



Each stage uses a different type of human input and produces a better model than the previous stage.

Stage 1: supervised fine-tuning (SFT)

Goal: start from a pretrained LLM and teach it to follow instructions.

How:

- human labellers write high-quality responses to a set of prompts
- the model is fine-tuned on these (prompt, response) pairs
- standard supervised learning — same as what we covered in the fine-tuning lecture

Output: a model that can follow instructions reasonably well.

(Refer to earlier lecture on LLM fine-tuning for details.)

Stage 1: supervised fine-tuning (SFT)

Goal: start from a pretrained LLM and teach it to follow instructions.

How:

- human labellers write high-quality responses to a set of prompts
- the model is fine-tuned on these (prompt, response) pairs
- standard supervised learning — same as what we covered in the fine-tuning lecture

Output: a model that can follow instructions reasonably well.

Limitation: labellers can only write so many examples. The model does not yet know *how good* its responses are — it just imitates.

(Refer to earlier lecture on LLM fine-tuning for details.)

Stage 2: Train a reward model

Collecting human preferences:

- take the SFT model and sample *multiple* responses to the same prompt
- show two responses (A and B) to a human labeller
- labeller picks which response is better

Stage 2: Train a reward model

Example:

Response A	**Response B**
"Paracetamol can cure all fevers and is completely safe in any dose."	"Paracetamol reduces fever but should be taken within recommended doses. Consult a doctor if unsure."
Human picks: B (more accurate and safe)	

This is repeated thousands of times across many prompts.
The result is a dataset of **preference pairs**: (x, y_w, y_l) where y_w is preferred over y_l.

Stage 2: Train a reward model

Goal: train a model $r(x, y)$ that assigns a higher score to better responses.

Think of it as a scoring function:

$$r(x, y) \in \mathbb{R} \qquad \text{(higher = better response)}$$

Stage 2: Train a reward model

Training:

- for each preference pair (x, y_w, y_l), the model should satisfy:

$$r(x, y_w) > r(x, y_l)$$

- the training loss penalises violations of this ordering

We will leave out additional details.

Output:

- a reward model that can score any (prompt, response) pair.

The reward model is typically initialised from the SFT model with an added scalar output head.

Stage 2: Train a reward model

Example:

Suppose the reward model has been trained. Given the prompt:

"What is the boiling point of water?"

Response	**Reward score**
"Water boils at 100°C at standard pressure."	$+2.8$
"Water boils when it gets hot."	$+0.4$
"Water boils at 50°C." (incorrect)	-1.9

The reward model has learned to prefer accurate, informative responses.

In Stage 3, we use these scores to improve the LLM.

Stage 3: using RL to improve the LLM

Now we have a reward signal. We can frame this as an RL problem:

RL concept	**In RLHF**
Agent	LLM
State	prompt (context so far)
Action	next token to generate
Policy $\pi_\theta(a\|s)$	LLM's token distribution
Reward	score from reward model $r(x, y)$

Stage 3: using RL to improve the LLM

The LLM is updated using policy gradient ideas (recall our earlier lecture):

$$\theta \leftarrow \theta + \alpha \cdot r(x, y) \cdot \nabla_\theta \log \pi_\theta(y \mid x)$$

In practice, a more stable algorithm called PPO is used.

(PPO details are beyond this course.)

Stage 3: using RL to improve the LLM

So far, everything seems okay. But, there is a problem :(

Reward hacking

If we only maximise the reward model score, the LLM will find shortcuts:

- produce very long, verbose responses (reward model may favour length)
- generate confident-sounding but incorrect content
- use flattery or filler phrases that fool the reward model

This is called reward hacking: the model exploits weaknesses in the reward model rather than becoming genuinely better.

Reward hacking

Fix: add a penalty that stops the LLM from drifting too far from the SFT model:

$$\text{objective} = \underbrace{r(x, y)}_{\text{reward}} - \beta \cdot \underbrace{\mathsf{KL}(\pi_\theta \| \pi_{\mathsf{SFT}})}_{\text{don't drift too far}}$$

The KL term keeps the LLM close to the well-behaved SFT model.

Direct preference optimisation (DPO)

Training a separate reward model and then running RL is complex and unstable.

DPO (Rafailov et al., 2023) is a simpler alternative:

- skip the reward model entirely
- directly optimise the LLM on preference pairs (x, y_w, y_l)
- one training stage instead of three

Direct preference optimisation (DPO)

Intuition: given that humans prefer y_w over y_l, increase the probability of y_w and decrease that of y_l — directly, without an intermediate reward score.

DPO has been shown to match or outperform RLHF in many settings.

(Loss function and theoretical details are beyond this course.)

Rafailov et al. (2023). Direct preference optimization. *NeurIPS*.

Real-world impact: InstructGPT and beyond

InstructGPT (Ouyang et al., 2022) was the first large-scale application of RLHF to LLMs:

- started from GPT-3 (175B parameters)
- applied the three-stage RLHF pipeline
- human evaluators strongly preferred InstructGPT over GPT-3 even though InstructGPT was $100\times$ smaller in a follow-up variant

Real-world impact: InstructGPT and beyond

RLHF is now central to how major AI systems are built:

System	**Key alignment technique**
ChatGPT (OpenAI)	RLHF
Claude (Anthropic)	RLHF + Constitutional AI
Gemini (Google)	RLHF variants
Llama 2 (Meta)	RLHF + rejection sampling

Ouyang et al. (2022). Training language models to follow instructions. *NeurIPS*.

Limitations of RLHF

RLHF is powerful but not perfect:

Annotation bias:

The reward model is only as good as the human labellers.

Different cultures, backgrounds, and opinions lead to inconsistent labels.

Reward hacking:

Even with the KL penalty, models find subtle ways to fool the reward model.

Limitations of RLHF

Sycophancy:

The model learns to say what humans *want* to hear, not what is true.

e.g., agreeing with a user's incorrect claim to get a positive rating.

Scalability:

Human labelling is expensive and slow.

AI feedback (RLAIF) is an active area of research as an alternative.